Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# Swagger & CORS

1.0 OAS3

/영남대/멋쟁이사자처럼 14기/김동민.json

Servers

/openapi

## default

POST /greeting/{name} Generate greeting

Generates a greeting message.

Parameters

Try it out

| Name | Description |
| --- | --- |
| name * required<br>string | Name of the person to greet. |

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# 목차

1.0 OAS3

## API 명세서

API 명세서란 ?
API 명세서를 써야하는 이유
API 명세서 구성 요소

## Swagger

Swagger란 ?
Swagger의 장점
Swagger의 단점
Swagger 실습 (Controller, DTO)

## API & Endpoint

API
REST API
Endpoint 설계 규칙

## API 명세서의 자동화

자동화의 장점
Spring Docs VS Swagger

## CORS

SOP
Origin의 구성
CORS 동작 원리
Spring boot에서 CORS 적용

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

1.0 OAS3

# API & Endpoint

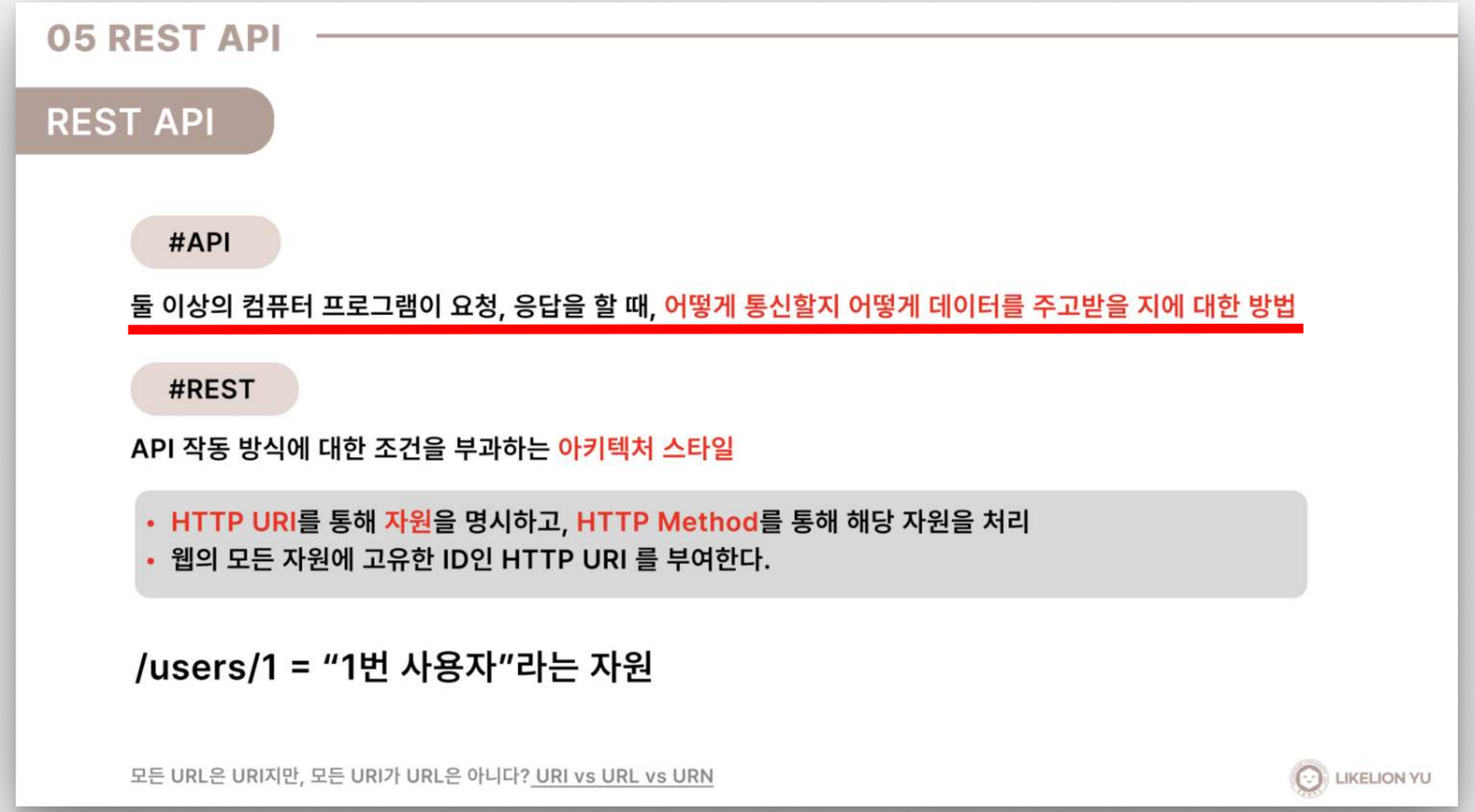
Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# API 란

1.0 OAS3



Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# REST API

1.0 OAS3

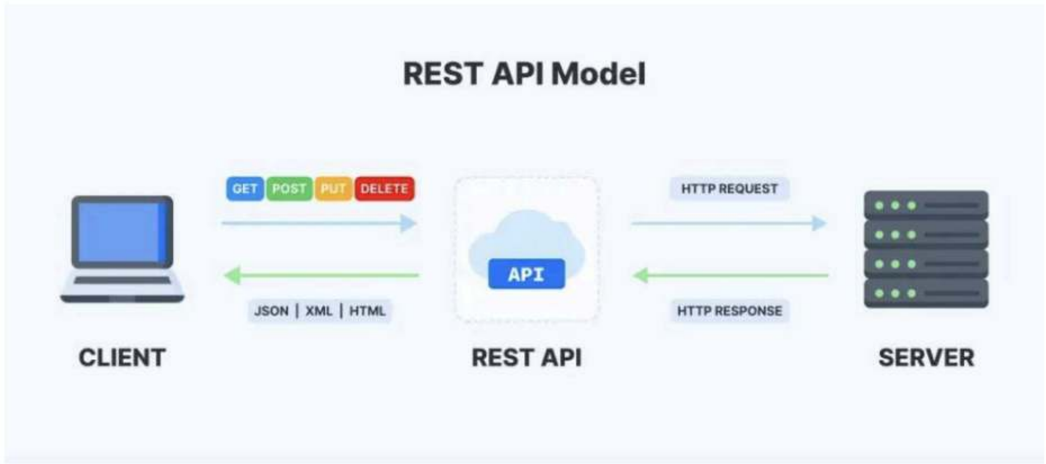
## 05 REST API

### REST API

#REST #API

웹 서비스와 클라이언트 간의 통신을 위해 REST 원칙을 따르는 API

전 슬라이드에 적힌 REST의 특징을 따라야겠죠?




LIKELION YU

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

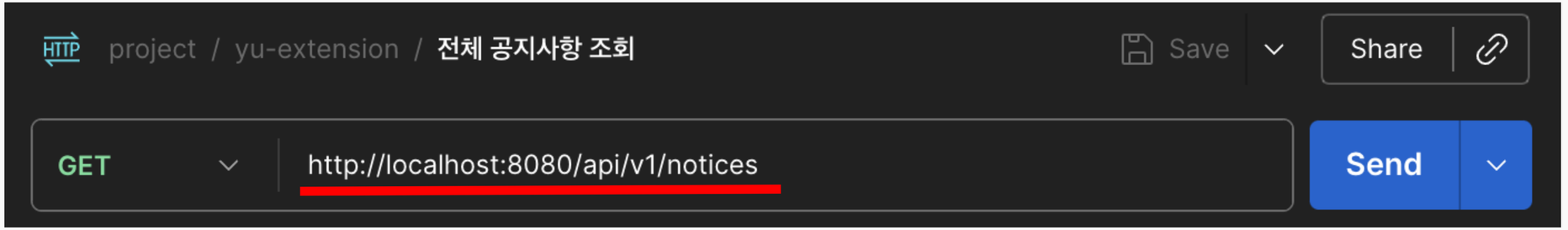
Explore

# EndPoint

1.0 OAS3

Endpoint는 클라이언트가 서버의 특정 기능에 접근하기 위한 구체적인 주소

**→ API 요청이 도착하는 목적지이다**



Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# API 명세서

1.0 OAS3

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# API 명세서

1.0 OAS3

API가 어떻게 이루어져 있는지 자세히 적어놓은 문서

- 엔드포인트
- HTTP 메소드
- 요청/응답 형식(JSON/XML)
- 헤더
- 파라미터 등을 기술한 문서

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# 그럼 왜 API 명세서를 써야할까 ?

그냥 메세지로 프론트한테 필요할 때 말해주면 안되나 ?

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# 만약 명세서가 없다면 ?

프론트 💬 : "로그인 API 주소가 뭐예요?"
백엔드 👤 : "/api/login 이요"
프론트 💬 : "POST로 보내면 되나요?"
백엔드 👤 : "네"
프론트 💬 : "email이랑 password 보내면 되나요?"
백엔드 👤 : "네"
프론트 💬 : "응답은 어떻게 오나요?"
백엔드 👤 : "토큰이랑 유저 정보요"
프론트 💬 : "에러는요?"
백엔드 👤 : "400 뜨면 비밀번호 틀린거예요"

## API 명세서



**POST /api/login**

사용자 로그인

**Request Body:**

```
{
"email": "user@example.com",
"password": "password123"
}
```

**Success Response (200):**

```
{
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
"user": {
"id": 1,
"email": "user@example.com",
"name": "홍길동"
}
}
```

**Error Response (400):**

```
{
"error": "INVALID_CREDENTIALS",
"message": "이메일 또는 비밀번호가 올바르지 않습니다."
}
```

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

**POST /api/login**

사용자 로그인

**Request Body:**

```
{
"email": "user@example.com",
"password": "password123"
}
```

**Success Response (200):**

```
{
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
"user": {
"id": 1,
"email": "user@example.com",
"name": "홍길동"
}
}
```

**Error Response (400):**

```
{
"error": "INVALID_CREDENTIALS",
"message": "이메일 또는 비밀번호가 올바르지 않습니다."
}
```

## 1. 불필요한 커뮤니케이션 감소

"이 필드 필수인가요?"
"날짜 형식이 어떻게 되나요?"
"에러 코드가 뭘 의미하나요?"

하루에 몇 번씩 이런 질문을 주고받다 보면,
정작 중요한 개발에 집중할 시간이 부족해질 수 있다.

명세서가 있으면 개발자들이 스스로 찾아볼 수 있고
필요한 기술적 논의에만 시간을 쓸 수 있어 개발 효율이 올라간다.

## 2. 병렬 개발 가능

백엔드가 API를 완성할 때까지
프론트엔드가 기다려야 한다면
프로젝트 일정이 늘어질 수밖에 없다.

하지만 명세서가 미리 작성되어 있다면?

프론트엔드는 mock 데이터를 만들어
UI를 먼저 완성할 수 있다.

## 3. 실수와 버그 감소

프론트엔드는 userId로 보냈는데,
백엔드는 user_id를 기대함
날짜를 문자열로 보냈는데,
백엔드는 타임스탬프를 기대함
필수 필드를 빠뜨려서 400 에러 발생

이런 실수들은 개발 시간을 낭비하게 만들고,
디버깅에 많은 시간을 소요하게 한다.

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# API 명세서 구성요소

1.0 OAS3

## 1. 기본 정보

```
API 이름: 사용자 로그인
HTTP Method: POST
Endpoint: /api/auth/login
설명: 이메일과 비밀번호로 사용자 인증을 수행합니다.
```

## 2. 요청 헤더 (필요한 경우)

```
Content-Type: application/json
Authorization: Bearer {token}  // 인증이 필요한 API의 경우
```

## 3. 요청 본문 (Request Body)

```
{
  "email": "user@example.com",    // string, required, 이메일 형식
  "password": "password123"       // string, required, 최소 8자
}
```

각 필드에 대해 다음 정보를 명시하면 좋다:

타입 (string, number, boolean, array, object 등)
필수 여부 (required / optional)
제약 조건 (최소/최대 길이, 형식, 허용 값 등)
예시 값

## 4. 응답 형식 (Response)

성공 시 (200 OK):

```
{
  "success": true,
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "user": {
      "id": 1,
      "email": "user@example.com",
      "name": "홍길동",
      "createdAt": "2025-11-03T10:00:00Z"
    }
  }
}
```

실패 시 (400 Bad Request):

```
{
  "success": false,
  "error": {
    "code": "INVALID_CREDENTIALS",
    "message": "이메일 또는 비밀번호가 올바르지 않습니다."
  }
}
```

## 5. 에러 코드 정의

| 상태 코드 | 에러 코드 | 설명 |
|---|---|---|
| 400 | INVALID_CREDENTIALS | 이메일 또는 비밀번호 불일치 |
| 400 | VALIDATION_ERROR | 요청 데이터 형식 오류 |
| 401 | UNAUTHORIZED | 인증 토큰 없음 또는 만료 |
| 429 | RATE_LIMIT_EXCEEDED | 요청 횟수 제한 초과 |
| 500 | INTERNAL_SERVER_ERROR | 서버 내부 오류 |

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

1.0 OAS3

# API 명세서의 자동화

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# 자동화가 왜 필요할까

1.0 OAS3

## 시간 절약

문서 자동화를 사용하면 Controller, DTO, Annotation 정보를 기반으로 문서가 자동 생성되기 때문에 개발자는 반복적인 문서 작성 시간을 줄일 수 있다

## 정확성 보장

문서 자동화를 사용하면 Controller와 DTO 정보를 기반으로 문서가 생성되기 때문에 실제 코드와 문서 사이의 불일치를 줄일 수 있다.

## 일관성 유지

문서 자동화를 사용하면 일정한 형식으로 API 문서를 생성할 수 있다.

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# Swagger VS Spring REST Docs

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore




**REST API를 문서화하고 테스트할 수 있게 도와주는 도구**

빠르게 API 문서를 만들고 싶을 때

프론트엔드와 바로 협업해야 할 때

브라우저에서 API를 직접 테스트하고 싶을 때

입문자에게 API 구조를 쉽게 보여주고 싶을 때

실습 중심 강의를 진행할 때

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# Spring REST Docs

**테스트를 기반으로 API 문서를 생성하는 도구**
**→ 테스트 코드가 성공적으로 실행된 API를 기준으로 문서를 만든다.**

테스트 코드를 기반으로 신뢰성 높은 문서를 만들고 싶을 때

운영 환경에서 문서와 API의 정확성을 중요하게 볼 때

TDD 또는 테스트 중심 개발 문화를 가진 팀일 때

문서 자동화보다 문서의 검증 가능성이 더 중요할 때

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

1.0 OAS3

# Swagger

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# Swagger 란

1.0 OAS3

**단순히 문서를 만드는 도구가 아니라, API를 확인하고 직접 테스트할 수 있는 문서화 도구**

어떤 URL로 요청해야 하는가?

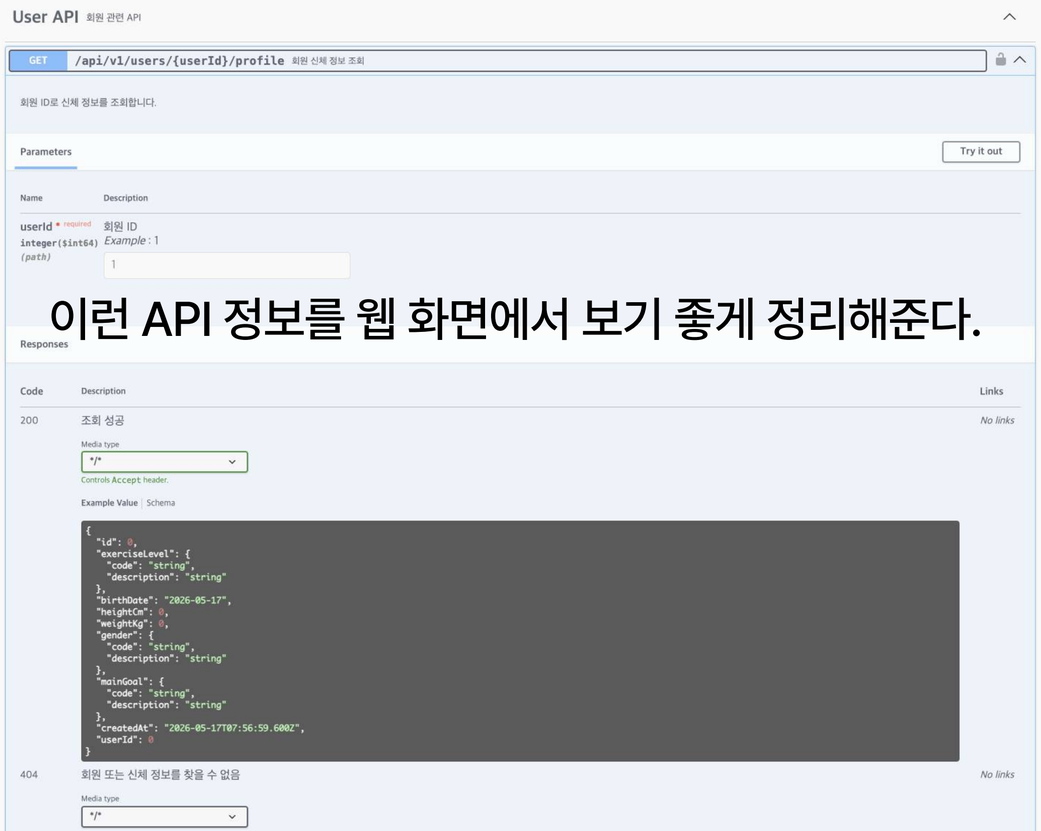
어떤 HTTP Method를 사용해야 하는가?

요청 Body에는 어떤 값을 넣어야 하는가?

응답 Body는 어떤 형태로 오는가?

성공/실패 시 상태 코드는 무엇인가?





Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# Swagger의 장점

1.0 OAS3

API 문서를 자동으로 생성

브라우저에서 API 테스트가 가능

API 구조를 한눈에 파악

**프론트엔드와 백엔드 협업이 쉬워진다**

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# Swagger의 단점

1.0 OAS3

## Annotation이 많아지면 코드가 복잡해질 수 있다

```
@Operation(
    summary = "회원가입",
    description = "사용자가 이메일, 비밀번호, 닉네임을 입력하여 회원가입한다."
)
@ApiResponses({
    @ApiResponse(responseCode = "201", description = "회원가입 성공"),
    @ApiResponse(responseCode = "400", description = "입력값 검증 실패"),
    @ApiResponse(responseCode = "409", description = "이메일 중복")
})
@PostMapping("/api/users")
public UserResponse createUser(@RequestBody UserCreateRequest request) {
    return userService.createUser(request);
}
```

## 테스트 코드를 보장하지는 않는다

Swagger UI에서 API를 직접 실행해볼 수는 있지만,
이것이 자동 테스트를 의미하지는 않음

Swagger는 API 문서화와 수동 테스트를 도와주는 도구이지,
테스트 코드를 대신 작성해주는 도구는 아니다.

예를 들어 Swagger에서 회원가입 API를 한 번 실행해봤다고 해서
다음 상황이 모두 검증되는 것은 아님

→ Swagger와 별개로 단위 테스트, 통합 테스트, API 테스트는 따로 작성해야 한다.

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# 실습

1.0 OAS3

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# CORS

1.0 OAS3

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# SOP

1.0 OAS3

**SOP는 같은 Origin끼리만 리소스 접근을 허용하는 브라우저 보안 정책 → 즉 브라우저 상에서, 같은 출처만 허용**
(Same-Origin Policy)

프론트엔드
http://localhost:3000

프론트엔드에서 백엔드 API를 호출

백엔드
http://localhost:8080

요청을 보낸 곳: http://localhost:3000
요청을 받는 곳: http://localhost:8080

둘은 Origin이 다르다.

그래서 브라우저는 이 요청을 기본적으로 제한하는데 이 정책이 SOP

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# Origin의 구성

1.0 OAS3

**Origin은 웹에서 출처를 구분하는 기준**

Protocol + Host + Port

프론트엔드

Protocol Host Port

**http://localhost:3000**

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# CORS

1.0 OAS3

**CORS는 다른 Origin 간 요청을 허용하기 위한 브라우저 보안 방식**
(Cross-Origin Resource Sharing)

SOP는 기본적으로 다른 Origin 요청을 막는다 → 실제 웹 서비스에서는 프론트엔드와 백엔드가 다른 Origin에 있는 경우가 많음

프론트엔드 서버 https://my-service.com
백엔드 서버 https://api.my-service.com

브라우저가 무조건 막아버리면 프론트엔드와 백엔드가 통신 X
이때 서버가 브라우저에게 다음과 같이 알려줌

**이 Origin에서 오는 요청은 허용**

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# CORS 설정법

1.0 OAS3

1.@CrossOrigin 사용 → @CrossOrigin은 특정 Controller나 특정 API에 CORS를 허용할 때 사용

2. WebMvcConfigurer 전역 설정 → Spring MVC 레벨에서 CORS를 설정

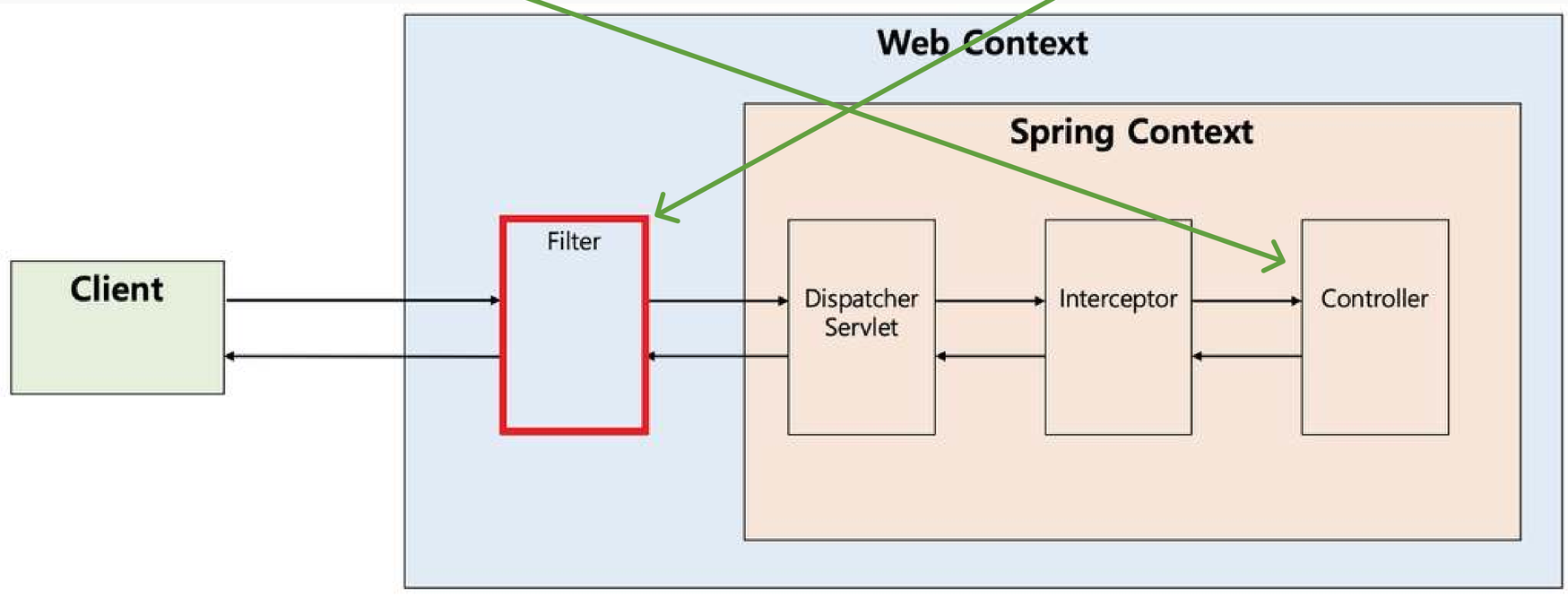
3. Spring Security CORS 설정 → Spring Security 필터 레벨에서 CORS를 설정

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# WebMvcConfigurer 전역 설정 vs Spring Security CORS 설정



Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# @CrossOrigin 사용

```java
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import yu.likelion14th.be_9th_lab.domains.auth.dto.request.SignInReqDto;
import yu.likelion14th.be_9th_lab.domains.auth.dto.request.SignUpReqDto;
import yu.likelion14th.be_9th_lab.domains.auth.dto.response.SignInResDto;
import yu.likelion14th.be_9th_lab.domains.auth.service.AuthService;

@CrossOrigin(origins = "http://localhost:5173")  JUNG CHOI *
@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {

    private final AuthService authService;


    @CrossOrigin(origins = "http://localhost:5173")  JUNG CHOI *
    @PostMapping("/sign-in")
    public ResponseEntity<?> signIn(@RequestBody SignInReqDto request) {
        SignInResDto response = authService.signIn(request);
        return ResponseEntity.ok().body(response);
    }
```

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# WebMvcConfigurer 전역 설정

```java
package com.example.demo.global.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
                // 허용할 프론트엔드 주소
                .allowedOrigins("http://localhost:5173")

                // 허용할 HTTP Method
                .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS")

                // 허용할 요청 Header
                .allowedHeaders("*")

                // 쿠키, 인증 정보 포함 요청 허용
                .allowCredentials(true)

                // Preflight 요청 캐시 시간
                .maxAge(3600);
    }
}
```

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# Spring Security CORS 설정

```java
@Configuration
public class CorsConfig {
    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();

        configuration.setAllowedOrigins(List.of(
                "http://localhost:5173",
                "http://127.0.0.1:5173"
        ));
        configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS")); // 허용할 HTTP 메서드
        configuration.setAllowedHeaders(List.of( e1: "*")); // 모든 요청 헤더 허용
        configuration.setExposedHeaders(List.of(HttpHeaders.AUTHORIZATION, HttpHeaders.SET_COOKIE)); // 응답에서 노출할 헤더
        configuration.setMaxAge(3600L); // Preflight 요청 캐시 시간 (초)
        configuration.setAllowCredentials(true); // 쿠키 포함 요청 허용

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration( pattern: "/**", configuration); // 모든 경로에 대해 설정 적용
        return source;
    }
}
```

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

1.0 OAS3

# 과제

Swagger
Supported by SMARTBEAR

/멋쟁이사자처럼/BE/10차시.json

Explore

# CORS 적용 후 테스트해보기

각 상황에 대한 결과를 캡쳐한 후, 개인톡으로 제출.

1. CORS 허용 테스트
2. CORS 실패 테스트

1. 요청하고자 하는 임의의 api 연결 → 2. ‘Headers’ 이동 → 3. Key에 Origin 입력 →4. Value에 테스트하고자 하는 도메인 입력

